

边改边学指南 · 修订版

LangChain + LangGraph 智能体上下文实施指南

Redis 6.2.6 热上下文优先写入 · Redis Streams 异步投递 · MongoDB 永久归档

适用项目	C:\work_learn\XinChuang_pc\agent_service
编写依据	当前仓库代码、LangChain / LangGraph 官方文档、Redis 官方文档
核心目标	删除自研快照、压缩节点和 Mongo-first 双写；让框架管理对话状态
文档日期	2026-08-29
实施方式	本文只给操作与代码，不直接修改项目文件

先看结论：这一版只保留四个组件

最终架构

LangGraph MessagesState 保存线程内消息；AsyncShallowRedisSaver 提供热检查点；LangChain trim_messages 在模型调用前裁剪瞬时输入；Redis Streams 把已完成轮次异步交给 MongoDBStore 永久归档。

职责	采用组件	本项目中的作用
会话短期记忆	MessagesState + add_messages	让 LangGraph 自动追加、替换和恢复 HumanMessage / AIMessage
Redis 热状态	AsyncShallowRedisSaver	按 thread_id 读写最新图状态；TTL 只控制热数据寿命
模型输入预算	trim_messages(token_counter='approximate')	只裁剪本次模型看到的内容，不改写已保存状态
Mongo 永久归档	MongoDBStore	按用户与会话 namespace 保存每个完成轮次；无 TTL
异步传输	Redis Streams + consumer group	API 只等待 XADD，不等待 MongoDB 写入

不再实现 load_context、compact_context、persist_context 三个自定义节点，不再维护 ContextSnapshot、revision、summary、estimated_tokens、facts 和 recent_entries。LangGraph 检查点已经负责线程状态；框架内置消息 reducer 已经负责消息合并；LangChain 已经提供消息裁剪。

上一版为什么需要推倒

- 当前 config.py 在类体中直接引用 self，导入时会触发错误；预算校验应位于 model_validator 内，但本方案直接删除整组自研压缩配置。
- context_repository.py 实现 MongoDB first、Redis second，与“Redis 热、Mongo 永久”的目标相反，并重复实现了框架已经提供的线程持久化语义。
- schemas/context.py 把对话状态重新建模为 ContextSnapshot，导致消息、摘要、事实、revision 与 LangGraph state 出现两套真相源。
- ModelGateway.complete() 只为自研摘要服务；删除压缩系统后应删除该方法，保持模型协议最小化。

- 当前 `graph.compile()` 没有 `checkpoint`，因此同一 `sessionId` 的下一调用不会自动恢复任何 `LangGraph state`。

删除范围

删除 `services/context_repository.py` 与 `schemas/context.py`；如果仓库中已经新增 `context_manager.py` 也一并删除。其余现有 SSE、取消、角色文件与 `ModelGateway` 流式接口继续保留。

最终运行时序

时序图（Redis 是在线路径，MongoDB 是异步永久归档）

请求进入

- > 用 `userId + sessionId` 生成稳定 `thread_id`
- > Redis 会话锁：同一会话只允许一个运行
- > Redis `checkpoint` 是否存在最新 `checkpoint`？
 - > 有：LangGraph 自动恢复 `messages`
 - > 无：MongoDBStore 读取永久轮次，`graph.aupdate_state()` 回填 Redis
- > `input_guard` 更新当前 `HumanMessage`（同一 `message id`，避免重复追加）
- > `generate` 调用 `trim_messages()` 生成本次模型输入
- > 模型流式输出，最终返回 `AIMessage`
- > `output_validate` 校验最后一条 `AIMessage`
- > LangGraph 自动把最新 `state` 写入 Redis
- > XADD 已完成轮次到 Redis Stream
- > API 发送 `done`（不等待 MongoDB）

后台消费者

- > `XREADGROUP / XAUTOCLAIM`
- > `MongoDBStore.aput(namespace, requestId, turn)`
- > 成功后 `XACK`

零号闸门：先确认 Redis 6.2.6 能否使用官方 Saver

必须先查模块

`langgraph-checkpoint-redis` 不只是使用普通 `GET/SET`。官方实现需要 `RedisJSON` 与 `RedisSearch`。Redis 8 已内置；Redis 6.2.6 必须额外加载模块或使用带这两个模块的 Redis Stack 发行版。缺模块时 `setup/asetup` 会失败。

虚拟机预检命令

```
# 在 Redis 虚拟机执行。密码只在终端临时输入，不要写进代码和文档。
redis-cli -h 192.168.100.128 -p 6379 -a '<Redis 密码>' INFO server
redis-cli -h 192.168.100.128 -p 6379 -a '<Redis 密码>' MODULE LIST

# 需要看到 Redis 版本为 6.2.6，并且模块列表包含：
# 1) ReJSON / RedisJSON
# 2) search / Redisearch

# Streams 与故障接管命令也要确认。
redis-cli -h 192.168.100.128 -p 6379 -a '<Redis 密码>' COMMAND INFO XADD XREADGROUP XAUTOCLAIM
```

检查结果	处理方式
两个模块都存在	继续本文推荐实现，使用 AsyncShallowRedisSaver。
模块缺失，但允许调整 Redis	给现有 6.2.6 安装匹配版本模块，或迁移到含模块的 Redis Stack；完成后重新预检。
模块缺失且基础设施不能改	不要自造 RedisSaver。改用 AsyncMongoDBSaver 作为 checkpointer；Redis 只保留 Stream。此分支不再具备 Redis 热读取优势。

本文后续代码按“模块已满足”编写。这样才能同时满足“尽量使用 LangChain/LangGraph 官方组件”和“Redis 在线热读写”。

第 1 步：清理上一版自研上下文

涉及文件 删除旧上下文文件；修改 config.py、models/gateway.py

1. 删除 agent_service/src/agent_service/services/context_repository.py。
2. 删除 agent_service/src/agent_service/schemas/context.py。
3. 若存在 services/context_manager.py，删除该文件。
4. 从 config.py 删除 context_soft_token_limit、context_hard_token_limit、context_recent_turns、context_summary_max_tokens、context_chars_per_token 及错误的类体 if self 校验。
5. 从 ModelGateway、MockModelGateway、OpenAIModelGateway 删除 complete()；它只服务于上一版自研摘要。
6. 从 README 中删除“增加压缩节点”和 Mongo-first Repository 的待办描述，改为本文架构。

models/gateway.py：删除 complete() 后的最小协议

```
class ModelGateway(Protocol):
    """ workflow 只依赖最终回答所需的流式模型能力。 """

    @property
    def model_name(self) -> str:
        """ 返回实际模型名称，供 done 事件和日志使用。 """

    def stream(
        self,
        messages: Sequence[BaseMessage],
        *,
        max_output_tokens: int,
    ) -> AsyncIterator[str]:
        """ 按增量文本输出模型结果。 """
```

第 2 步：增加官方持久化依赖

涉及文件 `agent_service/pyproject.toml`、`uv.lock`

依赖安装与导入检查

```
# 在 agent_service 根目录执行。uv 会同时更新 pyproject.toml 和 uv.lock。
uv add "langgraph-checkpoint-redis>=0.5,<0.6" "langgraph-store-mongodb>=0.3,<0.4"

# 现有 redis 与 pymongo 仍由依赖解析器保留；不要手工编辑 uv.lock。
uv run python -c "from langgraph.checkpoint.redis.ashallow import AsyncShallowRedisSaver; from
langgraph.store.mongodb import MongoDBStore; print('context integrations ok!)"
uv run ruff check .
uv run pytest
```

`AsyncShallowRedisSaver` 只保留每个 thread 的最新 checkpoint，正好对应“热上下文”而不是时间旅行审计；完整历史交给 `MongoDBStore`。若未来确实需要 `LangGraph time travel`，再把它替换为 `AsyncRedisSaver`，不需要改 state 和 workflow。

第 3 步：把配置缩减到真正需要的字段

涉及文件 `agent_service/src/agent_service/config.py`、`.env.example`

`config.py`：新的上下文配置片段

```
# config.py: 加入 Settings 类。这里只保留连接、TTL、回放上限和会话锁超时。
context_storage_enabled: bool = True

# Redis URL 包含密码，使用 SecretStr 防止 repr / 日志直接泄露。
redis_url: SecretStr | None = None
```

```

context_redis_ttl_minutes: int = Field(default=10080, ge=10) # 7 天
context_archive_stream: str = "xc:agent:context:archive:v1"
context_archive_group: str = "mongo-archive-v1"
context_archive_consumer: str = "mongo-archive-1"
context_rehydrate_turn_limit: int = Field(default=200, ge=1, le=2000)
session_lock_timeout_seconds: int = Field(default=180, ge=30)

# MongoDBStore 不配置 TTL, 因此它保存的是永久轮次。
mongodb_uri: SecretStr | None = None
mongodb_database: str = "xinchuang_agent_context"
mongodb_context_collection: str = "conversation_turns"

# trim_messages 使用框架的 approximate token counter; 这里只声明输入上限。
model_max_input_tokens: int = Field(default=8000, ge=1024)

@model_validator(mode="after")
def validate_runtime_settings(self) -> "Settings":
    """只校验真实运行必须具备的连接信息。测试可关闭持久化。"""

    if self.model_provider == "openai":
        if self.openai_api_key is None or not self.openai_api_key.get_secret_value():
            raise ValueError("AGENT_MODEL_PROVIDER=openai 时必须配置 AGENT_OPENAI_API_KEY")
        if self.model_name == "mock-model":
            raise ValueError("真实模型模式必须配置实际的 AGENT_MODEL_NAME")

    if self.context_storage_enabled:
        if self.redis_url is None or not self.redis_url.get_secret_value():
            raise ValueError("启用上下文持久化时必须配置 AGENT_REDIS_URL")
        if self.mongodb_uri is None or not self.mongodb_uri.get_secret_value():
            raise ValueError("启用上下文持久化时必须配置 AGENT_MONGODB_URI")

    return self

```

.env.example

```

# .env.example: 不要提交真实密码。
AGENT_CONTEXT_STORAGE_ENABLED=true
AGENT_REDIS_URL=redis://:<Redis 密码>@192.168.100.128:6379/4
AGENT_CONTEXT_REDIS_TTL_MINUTES=10080
AGENT_CONTEXT_ARCHIVE_STREAM=xc:agent:context:archive:v1
AGENT_CONTEXT_ARCHIVE_GROUP=mongo-archive-v1
AGENT_CONTEXT_ARCHIVE_CONSUMER=mongo-archive-1
AGENT_CONTEXT_REHYDRATE_TURN_LIMIT=200
AGENT_SESSION_LOCK_TIMEOUT_SECONDS=180

AGENT_MONGODB_URI=mongodb://<用户名>:<密码>@192.168.100.128:27017/xinchuang_agent_context?authSource=admin
AGENT_MONGODB_DATABASE=xinchuang_agent_context
AGENT_MONGODB_CONTEXT_COLLECTION=conversation_turns
AGENT_MODEL_MAX_INPUT_TOKENS=8000

```

第 4 步：让 LangGraph State 只保存消息

涉及文件 `agent_service/src/agent_service/graph/state.py`

graph/state.py：完整替换内容

```
"""LangGraph 的线程状态与单次运行上下文。"""

from dataclasses import dataclass

from langgraph.graph import MessagesState

class AgentState(MessagesState):
    """只持久化对话消息。

    MessagesState 已为 messages 字段配置 add_messages reducer：
    - 新 message id 会追加；
    - 相同 message id 会替换；
    - checkpointer 会按 thread_id 自动保存和恢复。
    """

    @dataclass(frozen=True, slots=True)
    class AgentRunContext:
        """本次调用需要、但不应该写入长期 state 的运行参数。"""

        request_id: int
        user_id: int
        session_id: int
        system_prompt: str
        model_route: str
        max_output_tokens: int
        max_input_tokens: int
```

为什么 requestId 不放 State

运行参数属于 Runtime Context，不是短期记忆。把它们放进 State 会让每个 checkpoint 都重复保存角色提示词、路由和预算；LangGraph 的 context_schema 正是为这类依赖设计的。

第 5 步：用 MessagesState 与 trim_messages 改造图

涉及文件 `agent_service/src/agent_service/graph/workflow.py`

graph/workflow.py：完整替换内容

```

"""基于 LangGraph 官方消息状态的最小智能体图。"""

from collections.abc import Callable

from langchain_core.messages import AIMessage, HumanMessage, SystemMessage, trim_messages
from langgraph.checkpoint.base import BaseCheckpointSaver
from langgraph.graph import END, START, StateGraph
from langgraph.graph.state import CompiledStateGraph
from langgraph.runtime import Runtime

from agent_service.core.cancellation import CancellationRegistry
from agent_service.core.exceptions import AgentCancelledError, EmptyModelOutputError
from agent_service.graph.state import AgentRunContext, AgentState
from agent_service.models.gateway import ModelGateway

def build_agent_graph(
    model_gateway: ModelGateway,
    cancellation_registry: CancellationRegistry,
    checkpointer: BaseCheckpointSaver,
) -> CompiledStateGraph:
    """创建图，并把 Redis/InMemory checkpointer 交给 LangGraph 管理。"""

    async def input_guard(
        state: AgentState,
        runtime: Runtime[AgentRunContext],
    ) -> dict[str, list[HumanMessage]]:
        """规范化本轮用户消息，并使用相同 id 原位替换。"""

        runtime.stream_writer({
            "event": "status",
            "payload": {"stage": "safety", "message": "正在检查输入内容"},
        })
        last = state["messages"][-1]
        if not isinstance(last, HumanMessage):
            raise ValueError("输入状态的最后一条消息必须是 HumanMessage")

        normalized = " ".join(str(last.content).split())
        return {
            "messages": [HumanMessage(content=normalized, id=last.id)],
        }

    async def generate(
        state: AgentState,
        runtime: Runtime[AgentRunContext],
    ) -> dict[str, list[AIMessage]]:
        """裁剪本次模型输入并流式生成最终 AIMessage。"""

        context = runtime.context
        runtime.stream_writer({
            "event": "status",
            "payload": {"stage": "generation", "message": "正在生成回答"},
        })

        # trim_messages 只改变本次模型看到的瞬时上下文；Redis 中的完整热状态不变。
        model_messages = trim_messages(
            [SystemMessage(content=context.system_prompt), *state["messages"]],
            max_tokens=context.max_input_tokens,
            token_counter="approximate",
            strategy="last",

```

```

        start_on="human",
        include_system=True,
        allow_partial=False,
    )

    parts: list[str] = []
    async for text in model_gateway.stream(
        model_messages,
        max_output_tokens=context.max_output_tokens,
    ):
        if await cancellation_registry.is_cancelled(context.request_id):
            raise AgentCancelledError(f"requestId={context.request_id} 已取消")
        parts.append(text)
        runtime.stream_writer({"event": "delta", "payload": {"content": text}})

    answer = "".join(parts).strip()
    if not answer:
        raise EmptyModelOutputError("模型没有生成可展示内容")

    # 稳定 id 让相同 requestId 的重试替换原消息，而不是重复追加。
    return {
        "messages": [
            AIMessage(
                content=answer,
                id=f"request:{context.request_id}:assistant",
            )
        ]
    }

    async def output_validate(state: AgentState) -> dict:
        """校验最新消息；不再创建 persist_context 节点。"""

        last = state["messages"][-1]
        if not isinstance(last, AIMessage) or not str(last.content).strip():
            raise EmptyModelOutputError("回答校验失败：内容为空")
        return {}

    graph = StateGraph(AgentState, context_schema=AgentRunContext)
    graph.add_node("input_guard", input_guard)
    graph.add_node("generate", generate)
    graph.add_node("output_validate", output_validate)
    graph.add_edge(START, "input_guard")
    graph.add_edge("input_guard", "generate")
    graph.add_edge("generate", "output_validate")
    graph.add_edge("output_validate", END)
    return graph.compile(checkpointer=checkpointer)

AgentGraphFactory = Callable[
    [ModelGateway, CancellationRegistry, BaseCheckpointSaver],
    CompiledStateGraph,
]

```

第 6 步：实现最薄的归档桥接层

涉及文件 新增 `agent_service/src/agent_service/services/context_archive.py`

这层不再管理模型上下文，也不实现 checkpoint 格式。它只做两件事：把已完成的一问一答 XADD 到 Stream；当 Redis 没有热 checkpoint 时，从 MongoDBStore 读取轮次并转换回标准消息。

services/context_archive.py: 完整新增文件

```
"""Redis Streams -> MongoDBStore 的上下文归档边界。"""

import json
from datetime import UTC, datetime

from langchain_core.messages import AIMessage, BaseMessage, HumanMessage
from langgraph.store.base import BaseStore
from redis.asyncio import Redis

def conversation_namespace(user_id: int, session_id: int) -> tuple[str, ...]:
    """MongoDBStore 的层级 namespace; 用户与会话天然隔离。"""

    return ("users", str(user_id), "conversations", str(session_id), "turns")

class ContextArchive:
    """在线请求只发布事件; MongoDB 写入由独立消费者完成。"""

    def __init__(
        self,
        *,
        redis: Redis,
        store: BaseStore,
        stream_name: str,
        rehydrate_turn_limit: int,
    ) -> None:
        self._redis = redis
        self._store = store
        self._stream_name = stream_name
        self._rehydrate_turn_limit = rehydrate_turn_limit

    async def publish_completed_turn(
        self,
        *,
        request_id: int,
        user_id: int,
        session_id: int,
        user_text: str,
        assistant_text: str,
    ) -> str:
        """发布一条可重试、可幂等落库的完成轮次事件。"""

        event = {
            "schema_version": 1,
            "request_id": request_id,
            "user_id": user_id,
            "session_id": session_id,
            "user_text": user_text,
            "assistant_text": assistant_text,
            "created_at": datetime.now(UTC).isoformat(),
        }

        # 不设置 MAXLEN: 直接裁剪 Stream 可能删除仍在 Pending 中的消息体。
        return await self._redis.xadd(
```

```

        self._stream_name,
        {"event": json.dumps(event, ensure_ascii=False)},
    )

async def load_messages(
    self,
    *,
    user_id: int,
    session_id: int,
) -> list[BaseMessage]:
    """Redis 热状态不存在时, 从 Mongo 永久归档重建最近消息。"""

    items = await self._store.asearch(
        conversation_namespace(user_id, session_id),
        limit=self._rehydrate_turn_limit,
    )
    # Store 搜索顺序不作为业务顺序; 按归档时间与 requestId 显式排序。
    items.sort(key=lambda item: (
        str(item.value["created_at"]),
        int(item.value["request_id"]),
    ))

    messages: list[BaseMessage] = []
    for item in items:
        value = item.value
        request_id = int(value["request_id"])
        messages.extend([
            HumanMessage(
                content=str(value["user_text"]),
                id=f"request:{request_id}:user",
            ),
            AIMessage(
                content=str(value["assistant_text"]),
                id=f"request:{request_id}:assistant",
            ),
        ])
    return messages

```

第 7 步：在 Runtime 中完成回填、运行与 XADD

涉及文件 `agent_service/src/agent_service/services/agent_runtime.py`

services/agent_runtime.py: 核心替换代码

```

"""LangGraph 运行协调器。"""

from collections.abc import AsyncIterator
from typing import Any

from langchain_core.messages import AIMessage, HumanMessage
from langgraph.checkpoint.base import BaseCheckpointSaver
from langgraph.graph.state import CompiledStateGraph
from redis.asyncio import Redis

from agent_service.core.cancellation import CancellationRegistry
from agent_service.core.exceptions import AgentCancelledError, RunAlreadyActiveError
from agent_service.graph.state import AgentRunContext

```

```

from agent_service.graph.workflow import build_agent_graph
from agent_service.models.gateway import ModelGateway
from agent_service.schemas.chat import AgentEvent, ChatStreamRequest
from agent_service.schemas.role import RoleProfile
from agent_service.services.context_archive import ContextArchive
from agent_service.services.role_profile import RoleProfileProvider

class AgentRuntime:
    """协调 Redis checkpointer、LangGraph 流和 Mongo 异步归档。"""

    def __init__(
        self,
        model_gateway: ModelGateway,
        role_profile_provider: RoleProfileProvider,
        *,
        checkpointer: BaseCheckpointSaver,
        archive: ContextArchive,
        redis: Redis,
        model_max_input_tokens: int,
        session_lock_timeout_seconds: int,
        cancellation_registry: CancellationRegistry | None = None,
    ) -> None:
        self.model_gateway = model_gateway
        self.role_profile_provider = role_profile_provider
        self.cancellation_registry = cancellation_registry or CancellationRegistry()
        self.checkpointer = checkpointer
        self.archive = archive
        self.redis = redis
        self.model_max_input_tokens = model_max_input_tokens
        self.session_lock_timeout_seconds = session_lock_timeout_seconds
        self.graph: CompiledStateGraph = build_agent_graph(
            model_gateway,
            self.cancellation_registry,
            checkpointer,
        )

    def load_active_role(self) -> RoleProfile:
        return self.role_profile_provider.load()

    @staticmethod
    def _thread_id(user_id: int, session_id: int) -> str:
        """同一 sessionId 在不同用户下不能共享 checkpoint。"""

        return f"user:{user_id}:session:{session_id}"

    async def _rehydrate_if_needed(
        self,
        *,
        config: dict[str, Any],
        user_id: int,
        session_id: int,
    ) -> None:
        """仅在 Redis checkpointer 未命中时查询 MongoDBStore。"""

        if await self.checkpointer.aget_tuple(config) is not None:
            return
        messages = await self.archive.load_messages(
            user_id=user_id,
            session_id=session_id,
        )
        if messages:
            # 让 LangGraph 自己生成合法 checkpoint; 不拼 Redis key、不写内部格式。

```

```

        await self.graph.aupdate_state(config, {"messages": messages})

    async def stream(
        self,
        request: ChatStreamRequest,
        role_profile: RoleProfile,
    ) -> AsyncIterator[AgentEvent]:
        """执行一轮，并在返回前确认归档事件已经进入 Redis Stream。"""

        thread_id = self._thread_id(request.user_id, request.session_id)
        config = {"configurable": {"thread_id": thread_id}}
        lock = self.redis.lock(
            f"xc:agent:context:lock:v1:{thread_id}",
            timeout=self.session_lock_timeout_seconds,
            blocking_timeout=0,
        )
        if not await lock.acquire(blocking=False):
            raise RunAlreadyActiveError(f"sessionId={request.session_id} 已有运行")

        await self.cancellation_registry.register(request.request_id)
        final_state: dict[str, Any] | None = None
        try:
            await self._rehydrate_if_needed(
                config=config,
                user_id=request.user_id,
                session_id=request.session_id,
            )

            run_context = AgentRunContext(
                request_id=request.request_id,
                user_id=request.user_id,
                session_id=request.session_id,
                system_prompt=role_profile.system_prompt,
                model_route=request.policy.model_route,
                max_output_tokens=request.policy.max_output_tokens,
                max_input_tokens=self.model_max_input_tokens,
            )
            input_message = HumanMessage(
                content=request.message,
                id=f"request:{request.request_id}:user",
            )

            async for part in self.graph.astream(
                {"messages": [input_message]},
                config,
                context=run_context,
                stream_mode=["custom", "values"],
                version="v2",
            ):
                if await self.cancellation_registry.is_cancelled(request.request_id):
                    raise AgentCancelledError(f"requestId={request.request_id} 已取消")
                if part["type"] == "custom":
                    yield AgentEvent.model_validate(part["data"])
                elif part["type"] == "values":
                    final_state = part["data"]

            if final_state is None:
                raise RuntimeError("LangGraph 未返回最终状态")
            last = final_state["messages"][-1]
            if not isinstance(last, AIMessage):
                raise RuntimeError("LangGraph 最终状态缺少 AIMessage")

```

```

# 先完成 Redis checkpoint, 再把本轮写入 Redis Stream。
# API 路由只有在本方法结束后才发送 done, 因此不会把未入队轮次报告为成功。
await self.archive.publish_completed_turn(
    request_id=request.request_id,
    user_id=request.user_id,
    session_id=request.session_id,
    user_text=request.message,
    assistant_text=str(last.content),
)
finally:
    await self.cancellation_registry.finish(request.request_id)
    if await lock.owned():
        await lock.release()

async def cancel(self, request_id: int) -> bool:
    return await self.cancellation_registry.cancel(request_id)

```

同一会话为什么要锁

LangGraph checkpointer 负责保存, 不等于自动把两个并发 invoke 串行化。使用 redis-py 自带 Lock 只解决会话占用, 不重复实现上下文。锁超时必须大于模型超时并留出归档入队余量。

第 8 步：用 FastAPI lifespan 管理官方组件

涉及文件 `agent_service/src/agent_service/main.py`

main.py：生产持久化分支的完整骨架

```

"""FastAPI 应用工厂和资源生命周期。"""

from contextlib import asynccontextmanager
import logging

from fastapi import FastAPI
from langgraph.checkpoint.redis.ashallow import AsyncShallowRedisSaver
from langgraph.store.mongodb import MongoDBStore
from redis.asyncio import Redis

from agent_service import __version__
from agent_service.api.routes import chat, health
from agent_service.config import Settings, get_settings
from agent_service.models.gateway import ModelGateway, create_model_gateway
from agent_service.services.agent_runtime import AgentRuntime
from agent_service.services.context_archive import ContextArchive
from agent_service.services.role_profile import RoleProfileProvider

def create_app(
    *,
    settings: Settings | None = None,
    model_gateway: ModelGateway | None = None,
) -> FastAPI:
    resolved = settings or get_settings()

```

```

logging.basicConfig(
    level=getattr(logging, resolved.log_level.upper(), logging.INFO),
    format="%(asctime)s %(levelname)s %(name)s %(message)s",
)

@asynccontextmanager
async def lifespan(app: FastAPI):
    redis_url = resolved.redis_url.get_secret_value() # validator 已保证存在
    mongodb_uri = resolved.mongodb_uri.get_secret_value()

    # Saver 自己管理 Redis checkpoint key; 应用不再拼接或覆盖这些 key。
    async with AsyncShallowRedisSaver.from_conn_string(
        redis_url,
        ttl={
            "default_ttl": resolved.context_redis_ttl_minutes,
            "refresh_on_read": True,
        },
    ) as checkpointer:
        # 生产中更推荐部署步骤执行一次; 重复调用应是幂等的。
        await checkpointer.asetup()

        # Stream 和分布式锁使用独立 redis-py 客户端。
        redis = Redis.from_url(redis_url, decode_responses=True)
        await redis.ping()

        # MongoDBStore 是 LangGraph 官方 Store; 不配置 index, 也不设置 TTL。
        with MongoDBStore.from_conn_string(
            conn_string=mongodb_uri,
            db_name=resolved.mongodb_database,
            collection_name=resolved.mongodb_context_collection,
        ) as store:
            store.setup()
            archive = ContextArchive(
                redis=redis,
                store=store,
                stream_name=resolved.context_archive_stream,
                rehydrate_turn_limit=resolved.context_rehydrate_turn_limit,
            )
            app.state.agent_runtime = AgentRuntime(
                model_gateway or create_model_gateway(resolved),
                RoleProfileProvider(resolved.role_config_path),
                checkpointer=checkpointer,
                archive=archive,
                redis=redis,
                model_max_input_tokens=resolved.model_max_input_tokens,
                session_lock_timeout_seconds=resolved.session_lock_timeout_seconds,
            )
            try:
                yield
            finally:
                await redis.aclose()

app = FastAPI(
    title="XinChuang Agent Service",
    version=__version__,
    description="信创智能客服独立 LangChain/LangGraph 智能体服务",
    lifespan=lifespan,
)
app.state.settings = resolved
app.include_router(health.router, prefix="/internal/ai/v1")
app.include_router(chat.router, prefix="/internal/ai/v1")
return app

```

```
app = create_app()
```

测试分支

单元测试不要连接虚拟机。create_app 可再增加可选 checkpointer/archive/redis 注入参数；测试传入 InMemorySaver、InMemoryStore 与 FakeRedis。重点是测试替身遵循框架接口，而不是重新实现 ContextSnapshot。

第 9 步：增加 MongoDB 归档消费者

涉及文件 新增 workers/context_archive_worker.py、consumer_main.py；修改 pyproject.toml

workers/context_archive_worker.py：完整新增文件

```
"""把 Redis Stream 中的完成轮次等写入 MongoDBStore."""

import asyncio
import json
import logging

from langgraph.store.base import BaseStore
from redis.asyncio import Redis
from redis.exceptions import ResponseError

from agent_service.services.context_archive import conversation_namespace

logger = logging.getLogger(__name__)

class ContextArchiveWorker:
    def __init__(
        self,
        *,
        redis: Redis,
        store: BaseStore,
        stream: str,
        group: str,
        consumer: str,
    ) -> None:
        self.redis = redis
        self.store = store
        self.stream = stream
        self.group = group
        self.consumer = consumer

    async def ensure_group(self) -> None:
        """首次启动创建 consumer group；已存在时忽略 BUSYGROUP."""

        try:
            await self.redis.xgroup_create(
```

```

        self.stream,
        self.group,
        id="0-0",
        mkstream=True,
    )
except ResponseError as exc:
    if "BUSYGROUP" not in str(exc):
        raise

async def persist(self, message_id: str, fields: dict[str, str]) -> None:
    """requestId 是 Store key; 重复投递只覆盖同一个文档。"""

    event = json.loads(fields["event"])
    namespace = conversation_namespace(
        int(event["user_id"]),
        int(event["session_id"]),
    )
    await self.store.aput(
        namespace,
        str(event["request_id"]),
        event,
        index=False,
    )
    # MongoDBStore 成功后才能 ACK; 失败时消息继续留在 Pending。
    await self.redis.xack(self.stream, self.group, message_id)

async def reclaim_stale(self) -> None:
    """接管崩溃消费者留下的 Pending 消息。

    Redis 6.2 的 XAUTOCLAIM 返回前两个元素; Redis 7 才增加已删除 ID 列表,
    因此这里只读取 result[0] 和 result[1], 兼容项目指定版本。
    """

    cursor = "0-0"
    while True:
        result = await self.redis.xautoclaim(
            self.stream,
            self.group,
            self.consumer,
            min_idle_time=60_000,
            start_id=cursor,
            count=50,
        )
        cursor, messages = result[0], result[1]
        for message_id, fields in messages:
            await self.persist(message_id, fields)
        if cursor in ("0-0", b"0-0"):
            return

async def run(self) -> None:
    await self.ensure_group()
    await self.reclaim_stale()
    while True:
        batches = await self.redis.xreadgroup(
            self.group,
            self.consumer,
            {self.stream: ">"},
            count=50,
            block=5_000,
        )
        for _stream_name, messages in batches:
            for message_id, fields in messages:
                try:

```

```

        await self.persist(message_id, fields)
    except Exception:
        # 不 ACK; 记录后由本消费者重试或由 XAUTOCLAIM 接管。
        logger.exception("上下文归档失败, messageId=%s", message_id)
        await asyncio.sleep(1)

```

consumer_main.py: 完整新增文件

```

"""MongoDB 上下文归档消费者进程入口。"""

import asyncio

from langgraph.store.mongodb import MongoDBStore
from redis.asyncio import Redis

from agent_service.config import get_settings
from agent_service.workers.context_archive_worker import ContextArchiveWorker

async def main() -> None:
    settings = get_settings()
    redis_url = settings.redis_url.get_secret_value()
    mongodb_uri = settings.mongodb_uri.get_secret_value()
    redis = Redis.from_url(redis_url, decode_responses=True)
    try:
        with MongoDBStore.from_conn_string(
            conn_string=mongodb_uri,
            db_name=settings.mongodb_database,
            collection_name=settings.mongodb_context_collection,
        ) as store:
            store.setup()
            worker = ContextArchiveWorker(
                redis=redis,
                store=store,
                stream=settings.context_archive_stream,
                group=settings.context_archive_group,
                consumer=settings.context_archive_consumer,
            )
            await worker.run()
    finally:
        await redis.aclose()

def run() -> None:
    asyncio.run(main())

```

pyproject.toml: 增加消费者入口, 不删除现有入口

```

[project.scripts]
agent-service = "agent_service.cli:main"
agent-context-consumer = "agent_service.consumer_main:run"

```

第 10 步：补齐错误映射与测试

涉及文件 `api/routes/chat.py`、`tests/`

`chat.py` 保持现有 SSE 结构。只需把 Redis/Mongo/归档队列故障映射为明确错误，不再沿用 `ContextRevisionConflictError`、`ContextTooLargeError`。

`api/routes/chat.py`：错误语义

```
# chat.py: 示意新增的领域错误分支。
except RunAlreadyActiveError:
    yield frame("error", {
        "code": "SESSION_RUN_ALREADY_ACTIVE",
        "message": "该会话已有请求正在执行",
        "retryable": True,
    })
except ContextArchiveUnavailableError:
    yield frame("error", {
        "code": "CONTEXT_ARCHIVE_UNAVAILABLE",
        "message": "上下文归档队列暂时不可用",
        "retryable": True,
    })
```

测试	验证点
同 thread 两轮对话	第二轮 state 自动包含第一轮 HumanMessage 与 AIMessage，不调用自研 load_context。
相同 requestId 重试	稳定 message id 不增加重复消息；Mongo Store 同 key 覆盖。
不同用户相同 sessionId	thread_id 与 Mongo namespace 都隔离。
Redis miss 回填	InMemorySaver 为空时，从 InMemoryStore 取轮次并通过 aupdate_state 重建。
trim_messages	模型收到系统消息与最近完整对话；热 state 仍保留全部测试消息。
消费者幂等	同一 Stream 消息处理两次，Store 中仍只有一个 requestId key。
Mongo 写失败	不 XACK，消息保持 Pending。
取消/空输出	不发布 completed turn；Mongo 中没有半截回答。
同会话并发	第二个请求拿不到 Redis Lock，返回 SESSION_RUN_ALREADY_ACTIVE。

`tests/conftest.py`：离线测试原则

```
# conftest.py 中测试配置必须显式关闭外部持久化，避免读取开发者 .env。
settings = Settings()
```

```

    environment="test",
    model_provider="mock",
    model_name="mock-model",
    internal_auth_enabled=False,
    context_storage_enabled=False,
)

# 建议测试注入：
# checkpointer = InMemorySaver()
# store = InMemoryStore()
# redis = FakeRedis (只实现 lock/xadd 或使用项目现有 fake)

```

第 11 步：部署与验证

涉及文件 Redis 虚拟机、MongoDB、两个 Python 进程

启动与 Redis 可靠性检查

```

# Redis 6.2.6 既保存热 checkpoint，又暂存未落 Mongo 的 Stream 消息。
# 因此必须开启持久化并禁止内存淘汰未归档数据。
redis-cli -h 192.168.100.128 -p 6379 -a '<Redis 密码>' CONFIG GET appendonly appendfsync maxmemory-policy

# 推荐结果：
# appendonly = yes
# appendfsync = everysec
# maxmemory-policy = noeviction

# 终端 1: API / LangGraph
uv run agent-service

# 终端 2: Redis Streams -> MongoDBStore
uv run agent-context-consumer

```

运行后检查

```

# 检查消费者组与积压。
redis-cli -h 192.168.100.128 -p 6379 -a '<Redis 密码>' XINFO GROUPS xc:agent:context:archive:v1
redis-cli -h 192.168.100.128 -p 6379 -a '<Redis 密码>' XPENDING xc:agent:context:archive:v1 mongo-archive-v1

# MongoDB 中检查永久归档（集合名由 MongoDBStore 版本决定具体内部字段）。
mongosh '<MongoDB URI>' --eval
'db.getSiblingDB("xinchuang_agent_context").conversation_turns.countDocuments({})'

```

不要用 MAXLEN 随便裁剪

Stream 中可能存在尚未 ACK 的消息。Redis 的 Stream 裁剪可删除 Pending 消息体，随后消费者只剩 ID 而无法归档。第一版先监控长度；只有设计了“已确认安全水位”后再做 MINID 裁剪。

一致性语义：这一版明确保证什么

场景	行为与保证
正常请求	LangGraph 先完成 Redis checkpoint；随后 XADD；XADD 成功后 API 才发送 done。
MongoDB 短暂不可用	API 仍可继续使用 Redis 热上下文；消费者不 ACK，恢复后重试。
Redis 热 checkpoint 过期	下一次请求从 MongoDBStore 读取永久轮次，并由 graph.aupdate_state 回填官方 saver。
Redis 整体不可用	在线图无法可靠 checkpoint，也无法 XADD；请求应失败，而不是无状态继续生成。
消费者崩溃	Pending 保留；重启后 XAUTOCLAIM 接管，MongoDBStore 以 requestId 幂等覆盖。
同会话并发	Redis Lock 拒绝第二个运行，避免两个分支同时更新同一 thread。
用户取消	不归档半截 AI 输出；Redis 可能保留上一个成功 checkpoint。

MongoDB 是最终永久记录，不是每个在线请求的同步真相源；Redis 是在线上下文与异步队列的当前真相源。二者通过 at-least-once Stream 传输和 requestId 幂等写入实现最终一致。

验收清单

- 项目中不存在 ContextSnapshot、ContextManager、load_context、compact_context、persist_context。
- graph.compile(checkpointer=...) 已启用，thread_id 同时包含 userId 与 sessionId。
- 连续两轮同会话对话时，第二轮自动使用第一轮消息。
- 模型输入超限时由 trim_messages 裁剪；没有自研字符/Token 估算器和摘要模型接口。
- API 完成后 Redis Stream 出现事件；MongoDB 停止期间 Pending 增加但聊天热状态仍可用。
- MongoDB 恢复并启动消费者后，Pending 归零且永久 Store 包含对应 requestId。
- 删除 Redis 热 checkpoint 后再发请求，Mongo 历史被回填并继续对话。

- 重复消费同一事件不会产生重复永久轮次。
- 不同用户使用相同 sessionId 不会互相读取上下文。
- 同一 sessionId 并发请求只有一个获得锁。
- `uv run ruff check .` 与 `uv run pytest` 全部通过。

第一版明确不做

- 不做自研摘要、事实抽取、revision 乐观锁、双写 Repository。
- 不为 MongoDB 保存 LangGraph 每个 super-step 的完整 checkpoint 历史；Mongo 只永久归档完成轮次。
- 不在第一版引入 Kafka、RabbitMQ 或额外消息系统；Redis 6.2.6 已具备 Streams 与 XAUTOCLAIM。
- 不把 MySQL chat_message 作为模型上下文真相源；它仍只服务于前端展示与业务审计。
- 不自动裁剪 Stream，不在未验证安全水位前使用 MAXLEN。

以后什么时候才需要摘要

只有真实长会话证明“只保留最近消息”损失了必要信息时，再把模型节点迁移到 LangChain create_agent，并加入官方 SummarizationMiddleware。不要先为尚未出现的问题保留 complete()、summary 字段和 compact_context 节点。

仅供未来演进的官方组件示例

```
# 未来可选，不属于第一版：使用 LangChain 内置中间件，不自写摘要节点。
from langchain.agents.middleware import SummarizationMiddleware

middleware = [
    SummarizationMiddleware(
        model="<便宜的摘要模型>",
        trigger=("tokens", 6000),
        keep=("messages", 20),
    )
]
```

官方资料与版本说明

以下链接均在 2026-08-29 查阅。实施时应以项目 uv.lock 的实际解析版本与对应 API reference 为准；不要把博客中的旧 import 路径直接复制到项目。

- **LangGraph Persistence: checkpointer 管线状态, Store 管跨线程/应用数据:**
<https://docs.langchain.com/oss/python/langgraph/persistence>
- **LangGraph Memory: MessagesState、Redis/MongoDB checkpointer 示例:**
<https://docs.langchain.com/oss/python/langgraph/add-memory>
- **LangGraph Checkpointer Integrations:** <https://docs.langchain.com/oss/python/integrations/checkpointers/index>
- **LangGraph Redis 官方实现: 模块要求、AsyncRedisSaver、Shallow saver 与 TTL:** <https://github.com/redis-developer/langgraph-redis>
- **LangChain 内置 SummarizationMiddleware:** <https://docs.langchain.com/oss/python/langchain/middleware/built-in>
- **MongoDB 官方 LangGraph 集成: MongoDBSaver 与 MongoDBStore:** <https://www.mongodb.com/docs/atlas/ai-integrations/langgraph/>
- **Redis XAUTOCLAIM: 自 Redis 6.2.0 起可用:** <https://redis.io/docs/latest/commands/xautocclaim/>
- **Redis Streams 消费者组与故障接管:** <https://redis.io/docs/latest/develop/data-types/streams/>

实施前的最后一次确认

Go / No-Go

只有当 Redis 6.2.6 已确认加载 RedisJSON 与 RediSearch, 且 AOF / noeviction 策略满足队列可靠性要求时, 才执行推荐方案。否则先完成基础设施调整, 或明确接受 MongoDBSaver 同步在线路径的退化方案。